and the consequences of making a false-positive error versus a false-negative error. In the present example, we will arbitrarily declare that the error in detecting a target is *five* times as bad as the error in detecting a nontarget. In effect, this tells the network to do a better job with the targets, even if it hurts the performance of the nontargets.

Subroutine 3000 is the heart of the neural network strategy, the algorithm for changing the weights on each iteration. We will use an analogy to explain the underlying mathematics. Consider the predicament of a military paratrooper dropped behind enemy lines. He parachutes to the ground in unfamiliar territory, only to find it is so dark he can't see more than a few feet away. His orders are to proceed to the bottom of the nearest valley to begin the remainder of his mission. The problem is, without being able to see more than a few feet, how does he make his way to the valley floor? Put another way, he needs an algorithm to adjust his x and y position on the earth's surface in order to *minimize* his elevation. This is analogous to the problem of adjusting the neural network weights, such that the network's error, ESUM, is minimized.

We will look at two algorithms to solve this problem: **evolution** and **steepest descent**. In evolution, the paratrooper takes a flying jump in some random direction. If the new elevation is *higher* than the previous, he curses and returns to his starting location, where he tries again. If the new elevation is *lower*, he feels a measure of success, and repeats the process from the new location. Eventually he will reach the bottom of the valley, although in a very inefficient and haphazard path. This method is called *evolution* because it is the same type of algorithm employed by nature in biological evolution. Each new generation of a species has random variations from the previous. If these differences are of benefit to the species, they are more likely to be retained and passed to the *next* generation. This is a result of the improvement allowing the animal to receive more food, escape its enemies, produce more offspring, etc. If the new trait is detrimental, the disadvantaged animal becomes lunch for some predator, and the variation is discarded. In this sense, each new generation is an iteration of the evolutionary optimization procedure.

When evolution is used as the training algorithm, each weight in the neural network is slightly changed by adding the value from a random number generator. If the modified weights make a better network (i.e., a lower value for ESUM), the changes are retained, otherwise they are discarded. While this works, it is very slow in **converging**. This is the jargon used to describe that continual improvement is being made toward an optimal solution (the bottom of the valley). In simpler terms, the program is going to need days to reach a solution, rather than minutes or hours.

Fortunately, the *steepest descent* algorithm is much faster. This is how the paratrooper would naturally respond: evaluate which way is *downhill*, and move in that direction. Think about the situation this way. The paratrooper can move one step to the north, and record the change in elevation. After returning to his original position, he can take one step to the east, and